Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №4

«Функциональные возможности языка Python Часть 2»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Сердюков А.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Реализовать декоратор, два контекстных менеджера и конвейер обработки вакансий. Работа содержит задачи 5–7 общего задания №3–4.

Задача 5 — print_result, печатающий имя функции и результат, но сохраняющий возвращаемое значение. Задача 6 — cm_timer_1 и cm_timer_2, измеряющие время блока двумя способами. Задача 7 — цепочка f1–f4 для отбора профессий, добавления требования Python и генерации зарплаты.

Описание решения

Декоратор принимает args и kwargs, вызывает исходную функцию один раз и сохраняет её метаданные через wraps. Списки выводятся построчно, словари — парами ключ = значение. Первый таймер — класс с __enter__/__exit__, второй — генератор с contextmanager. Оба измеряют perf_counter и выводят время даже при исключении, сохраняя само исключение.

f1 извлекает job-name, убирает повторы без учёта регистра и сортирует. f2 применяет filter для префикса «программист». f3 через map добавляет «с опытом Python». f4 соединяет профессии и случайные зарплаты 100000–200000 через zip. Функции f1–f3 однострочные, f4 занимает две строки. Параметр --seed 22 обеспечивает воспроизводимость зарплат.

Обработаны все 4635 записей из учебного data_light.json. В репозиторий включена проекция на job-name, сохраняющая порядок и повторы; исходная ссылка и SHA-256 указаны в data/README.md. Полный вывод находится в results/full_data.txt. Короткий demo.json составлен отдельно для наглядной проверки. Сгенерированные зарплаты — учебные данные.

Порядок запуска

Python 3.10 или новее. Из корня репозитория создать виртуальное окружение и установить зависимости командой python -m pip install -r requirements.txt.

```
python lab_04/lab_python_fp/print_result.py
python lab_04/lab_python_fp/cm_timer.py
python lab_04/main.py lab_04/data/demo.json --seed 22
python lab_04/main.py lab_04/data/jobs.json --seed 22
```

Результаты выполнения

```
$ python lab_04/lab_python_fp/print_result.py
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2

$ python lab_04/lab_python_fp/cm_timer.py
time: 0.020114
time: 0.020105

$ python lab_04/main.py lab_04/data/demo.json --seed 22
f1
Аналитик
Программист C#
Программист Python
f2
Программист C#
Программист Python
f3
Программист C# с опытом Python
Программист Python с опытом Python
f4
Программист C# с опытом Python, зарплата 118398 руб.
Программист Python с опытом Python, зарплата 131800 руб.
time: 0.000040

Итог тестового запуска (полный журнал tests.txt)
$ python -m pytest lab_04 -v --tb=short
4 passed in 0.05s
```

Рисунок 1 — Вывод фактического запуска программы

Экранная форма сформирована из сохранённого stdout. Полные журналы: results/demo.txt и results/tests.txt.

Проверка и выводы

Для этой работы успешно выполнено проверок: 4. Общий набор всех шести работ содержит 61 тест; полный прогон завершился без ошибок. Версии среды и журналы сохранены в validation/ и results/tests.txt.

Конвейер объединяет генераторы, декоратор и менеджер контекста. Проверены результат цепочки, сохранение аргументов и результата декоратора, а также вывод времени и распространение исключений. Вместе с работой №3 выполнены все семь задач функционального задания.

Источник задания

https://github.com/ugapanyuk/BKIT_2022/wiki/lab_python_fp

Текст программы

```
# Файл main.py

"""Запуск из любой рабочей директории с модулями лабораторной 3."""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "lab_03"))
from lab_python_fp.process_data import main


if __name__ == "__main__":
    main()


# Файл lab_python_fp/__init__.py

"""Вторая часть общего пакета лабораторных 3 и 4."""
from pkgutil import extend_path

__path__ = extend_path(__path__, __name__)


# Файл lab_python_fp/print_result.py

"""Задача 5. Декоратор сохраняет результат и метаданные функции."""
from functools import wraps


def print_result(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        result = function(*args, **kwargs)
        print(function.__name__)
        if isinstance(result, dict):
            for key, value in result.items():
                print(f"{key} = {value}")
        elif isinstance(result, list):
            for value in result:
                print(value)
        else:
            print(result)
        return result
    return wrapper


@print_result
def test_1():
    return 1


@print_result
def test_2():
    return "iu5"


@print_result
```

Текст программы Продолжение

```
def test_3():
    return {"a": 1, "b": 2}


@print_result
def test_4():
    return [1, 2]


if __name__ == "__main__":
    test_1()
    test_2()
    test_3()
    test_4()


# Файл lab_python_fp/cm_timer.py

"""Задача 6. Класс и генераторный контекстный менеджер."""
from contextlib import contextmanager
from time import perf_counter, sleep


class cm_timer_1:
    def __enter__(self):
        self.started = perf_counter()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print(f"time: {perf_counter() - self.started:.6f}")
        return False


@contextmanager
def cm_timer_2():
    started = perf_counter()
    try:
        yield
    finally:
        print(f"time: {perf_counter() - started:.6f}")


if __name__ == "__main__":
    with cm_timer_1():
        sleep(0.02)
    with cm_timer_2():
        sleep(0.02)


# Файл lab_python_fp/process_data.py

"""Задача 7. Конвейер обработки вакансий."""
import argparse
import json
import random
from lab_python_fp.field import field
```

Текст программы Продолжение

```
from lab_python_fp.gen_random import gen_random
from lab_python_fp.unique import Unique
from lab_python_fp.print_result import print_result
from lab_python_fp.cm_timer import cm_timer_1


@print_result
def f1(arg):
    return sorted(Unique(field(arg, "job-name"), ignore_case=True),
        key=str.casefold)


@print_result
def f2(arg):
    return list(filter(lambda title:
        title.casefold().startswith("программист"), arg))


@print_result
def f3(arg):
    return list(map(lambda title: title + " с опытом Python", arg))


@print_result
def f4(arg):
    return [f"{title}, зарплата {salary} руб." for title, salary in
            zip(arg, gen_random(len(arg), 100000, 200000))]


def main():
    parser = argparse.ArgumentParser(description="Обработка списка вакансий")
    parser.add_argument("path", help="JSON-файл со списком вакансий")
    parser.add_argument("--seed", type=int, default=None)
    args = parser.parse_args()
    if args.seed is not None:
        random.seed(args.seed)
    with open(args.path, encoding="utf-8") as source:
        data = json.load(source)
    if not isinstance(data, list):
        parser.error("JSON должен содержать список")
    with cm_timer_1():
        f4(f3(f2(f1(data))))


if __name__ == "__main__":
    main()


# Файл test_pipeline.py

import pytest
from unittest.mock import patch
from lab_python_fp.print_result import print_result
from lab_python_fp.cm_timer import cm_timer_1, cm_timer_2
from lab_python_fp.process_data import f1, f2, f3, f4
```

Текст программы Продолжение

```
def test_pipeline():
    data = [{"job-name": "Программист Python"}, {"job-name": "программист
        python"},
            {"job-name": "Аналитик"}, {}]
    with patch("lab_python_fp.gen_random.random.randint",
        return_value=150000):
        assert f4(f3(f2(f1(data)))) == [
            "Программист Python с опытом Python, зарплата 150000 руб."]


def test_decorator_preserves_return_and_arguments(capsys):
    @print_result
    def add(a, b=0):
        return {"sum": a + b}
    assert add(2, b=3) == {"sum": 5}
    assert add.__name__ == "add"
    assert capsys.readouterr().out == "add\nsum = 5\n"


@pytest.mark.parametrize("timer", [cm_timer_1, cm_timer_2])
def test_timers_log_and_propagate_exception(timer, capsys):
    with patch("lab_python_fp.cm_timer.perf_counter", side_effect=[10, 15.5]):
        with pytest.raises(RuntimeError):
            with timer():
                raise RuntimeError("failure")
    assert capsys.readouterr().out == "time: 5.500000\n"


# Файл data/demo.json

[
  {"job-name": "Программист Python"},
  {"job-name": "программист python"},
  {"job-name": "Аналитик"},
  {"job-name": "Программист C#"},
  {"job-name": null},
  {}
]
```